

Java Fundamentals for Data Structures & Algorithms

A Structured Guide from Beginner to Advanced

Prepared with Claude

July 2026

How to Use This Guide

Part I: Foundations of Java

1. Introduction to Java
2. Setting Up Your Environment
3. Java Syntax Basics
4. Variables and Data Types
5. Operators
6. Input and Output
7. Control Flow: Conditionals
8. Control Flow: Loops

Part II: Core Building Blocks for DSA

9. Arrays
10. Multi-Dimensional Arrays
11. Strings and StringBuilder
12. Methods and Parameter Passing
13. Recursion

Part III: Object-Oriented Programming for DSA

14. Classes and Objects
15. Constructors and the `this` Keyword
16. Encapsulation
17. Inheritance
18. Polymorphism
19. Abstraction: Abstract Classes and Interfaces

20. Generics

Part IV: The Java Collections Framework

21. Overview of Collections

22. List: ArrayList and LinkedList

23. Stack and Deque

24. Queue and PriorityQueue

25. Set: HashSet, LinkedHashSet, TreeSet

26. Map: HashMap, LinkedHashMap, TreeMap

27. Comparable and Comparator

Part V: Advanced Java for DSA

28. Exception Handling

29. Wrapper Classes and Autoboxing

30. Big-O Notation and Complexity in Java

31. Bit Manipulation

32. Lambda Expressions and Streams (Brief Introduction)

Part VI: Bridging Java to DSA Practice

33. Choosing the Right Data Structure

34. Implementing a Stack and Queue from Scratch

35. Worked DSA Problems in Java

36. Real-World Applications

Summary and Key Takeaways

Further Reading and References

How to Use This Guide

This guide takes you from your first line of Java to the language features you need to study Data Structures and Algorithms (DSA) with confidence. It is organized into six parts, each building on the last:

- **Part I — Foundations:** syntax, variables, operators, and control flow.
- **Part II — Core Building Blocks:** arrays, strings, methods, and recursion — the raw material of every data structure.
- **Part III — Object-Oriented Programming:** classes, inheritance, and generics, which let you build reusable, type-safe structures.
- **Part IV — The Collections Framework:** Java’s ready-made data structures (lists, stacks, queues, sets, maps) that you will use constantly in DSA work.
- **Part V — Advanced Java for DSA:** exceptions, complexity analysis, bit manipulation, and modern Java features.
- **Part VI — Bridging to Practice:** choosing the right structure, implementing your own, and worked problems.

Every chapter includes runnable code, and most include short exercises. Try each exercise before checking the discussion that follows it — the goal is to build muscle memory, not just recognition.

Prerequisites: none. This guide assumes no prior programming experience, though anyone with experience in another language will move quickly through Part I.

How code examples are formatted: all examples are complete or near-complete Java snippets. Where a full class wrapper (`public class Main { public static void main(String[] args) { ... } }`) is omitted for brevity, it is noted.

Part I: Foundations of Java

1. Introduction to Java

Java is a general-purpose, class-based, object-oriented programming language first released by Sun Microsystems in 1995 and now developed by Oracle. It is compiled to *bytecode* that runs on the Java Virtual Machine (JVM), which is what gives Java its “write once, run anywhere” portability: the same compiled `.class` file runs unmodified on Windows, Linux, or macOS, provided a compatible JVM is installed.

Oracle's official documentation describes the Java SE platform as letting developers build and deploy applications across desktops and servers, with an emphasis on portability, performance, and security (Oracle, *Java Documentation*, docs.oracle.com/en/java/).

Why Java for learning DSA?

- **Static typing** catches many mistakes at compile time, before you ever run the program — valuable when you are learning how data structures behave.
- **Automatic memory management** (garbage collection) means you can focus on algorithmic logic instead of manual memory bookkeeping, unlike C/C++.
- **The Java Collections Framework** ships a complete, well-documented, heavily optimized set of data structures (lists, sets, maps, queues) that mirror almost every structure taught in a standard DSA course.
- Java is one of the most widely used languages in technical interviews and university DSA courses, so skills transfer directly.

Key Java Concepts at a Glance

Concept	What it means
JDK	Java Development Kit — compiler, tools, and libraries used to <i>build</i> Java programs
JRE	Java Runtime Environment — what you need to <i>run</i> compiled Java programs (bundled inside the JDK since Java 11)
JVM	Java Virtual Machine — executes compiled bytecode
<code>.java</code> file	Human-readable source code
<code>.class</code> file	Compiled bytecode produced from a <code>.java</code> file

2. Setting Up Your Environment

1. Install a JDK (Java Development Kit). The official builds are available from Oracle's Java SE Downloads page or the open-source OpenJDK project.
2. Verify the install from a terminal:

```
java -version
javac -version
```

1. Write and run your first program.

```
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, DSA!");
    }
}
```

Compile and run:

```
javac HelloWorld.java
java HelloWorld
```

What just happened? `javac` compiled `HelloWorld.java` into `HelloWorld.class` (bytecode). The `java` command launched the JVM, which loaded that bytecode and executed the `main` method — the fixed entry point every standalone Java program must have, as documented in Oracle’s *Java Tutorials* trail on getting started (docs.oracle.com/javase/tutorial/getStarted/).

Tip: Most learners today write Java inside an IDE (IntelliJ IDEA, Eclipse, or VS Code with the Java extension pack) rather than a bare text editor. For short DSA practice snippets, an online JDK sandbox works fine too.

3. Java Syntax Basics

Every Java application is organized into **classes**, and every statement lives inside a method, which lives inside a class.

```
public class Main {
    public static void main(String[] args) {
        // A single-line comment
        /* A multi-line
           comment */
        System.out.println("Statements end with a semicolon;");
    }
}
```

Key syntax rules:

- Java is **case-sensitive**: `myValue` and `myvalue` are different identifiers.
- Every statement ends in a semicolon `;`.
- Code blocks are delimited by curly braces `{ }`.
- The filename must match the public class name exactly (`Main.java` for `class Main`).

Exercise 1.1: Write a program that prints your name, then on the next line prints the current year, using two separate `System.out.println` calls.

4. Variables and Data Types

Java is **statically typed**: every variable's type is fixed at compile time.

Primitive Types

Type	Size	Example	Typical DSA use
<code>int</code>	32-bit	<code>int count = 10;</code>	array indices, loop counters, most integer math
<code>long</code>	64-bit	<code>long big = 10_000_000_000L;</code>	values that overflow <code>int</code> (> ~2.1 billion)
<code>double</code>	64-bit floating point	<code>double avg = 3.14;</code>	averages, probabilities, floating results
<code>boolean</code>	true/false	<code>boolean found = false;</code>	flags, visited arrays in graph traversal
<code>char</code>	16-bit Unicode character	<code>char c = 'A';</code>	processing individual characters in strings
<code>byte</code> , <code>short</code>	8-bit, 16-bit	rarely used directly	memory-constrained scenarios

Reference Types

Everything that is not a primitive — `String`, arrays, and objects of any class — is a **reference type**. A reference variable holds a pointer to an object on the heap, not the object's data directly.

```
int age = 25;           // primitive
String name = "Alice"; // reference type
int[] scores = {90, 85, 77}; // reference type (array)
```

Declaring and Initializing

```
int x;           // declaration only – has no usable value yet
x = 5;           // assignment
int y = 10;      // declaration + initialization in one line
final int MAX = 100; // a constant: cannot be reassigned
```

Exercise 1.2: Declare an `int`, a `double`, a `boolean`, and a `String` variable describing a book (page count, price, “is available”, title). Print all four on one line separated by commas.

5. Operators

```
int a = 7, b = 2;

int sum      = a + b;    // 9
int difference = a - b;  // 5
int product   = a * b;   // 14
int quotient  = a / b;   // 3 (integer division truncates!)
int remainder = a % b;   // 1 (modulo – essential for cycling through array
indices)

double preciseQuotient = (double) a / b; // 3.5 – casting forces floating-point
division
```

Comparison and logical operators (used constantly in loop and recursion conditions):

```
a == b    // equality
a != b    // inequality
a > b, a < b, a >= b, a <= b

boolean bothPositive = (a > 0) && (b > 0); // logical AND
boolean eitherPositive = (a > 0) || (b > 0); // logical OR
boolean notPositive = !(a > 0);           // logical NOT
```

Increment/decrement, seen everywhere in loops:

```
int i = 0;
i++;    // post-increment, i becomes 1
++i;    // pre-increment, i becomes 2
i--;    // i becomes 1
```

Common DSA pitfall: integer division truncates. $7 / 2$ is 3 , not 3.5 . This matters when computing a midpoint in binary search: `int mid = low + (high - low) / 2;` deliberately relies on integer truncation and also avoids overflow that `(low + high) / 2` can cause with large indices.

6. Input and Output

```
import java.util.Scanner;

public class InputDemo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a number: ");
        int n = sc.nextInt();
        System.out.println("You entered: " + n);
        sc.close();
    }
}
```

`Scanner` is the standard class for reading console input, documented in the `java.util` package of the Java SE API (docs.oracle.com). Common methods: `nextInt()`, `nextLong()`, `nextDouble()`, `next()` (single token), and `nextLine()` (full line).

7. Control Flow: Conditionals

```
int score = 82;

if (score >= 90) {
    System.out.println("Grade: A");
} else if (score >= 75) {
    System.out.println("Grade: B");
} else {
    System.out.println("Grade: C or below");
}

// switch – useful when branching on one variable's exact value
int day = 3;
switch (day) {
    case 1 -> System.out.println("Monday");
    case 2 -> System.out.println("Tuesday");
    case 3 -> System.out.println("Wednesday");
    default -> System.out.println("Another day");
}
```

The arrow-style `switch` shown above is the modern form (Java 14+). The classic `case 3: ... break;` form still works and is common in existing codebases and older tutorials.

8. Control Flow: Loops

```
// for loop – best when you know the number of iterations
for (int i = 0; i < 5; i++) {
    System.out.println("i = " + i);
}

// while loop – best when the stopping condition isn't a simple counter
int n = 27;
int steps = 0;
while (n != 1) {
    n = (n % 2 == 0) ? n / 2 : 3 * n + 1; // Collatz step
    steps++;
}
System.out.println("Steps: " + steps);

// do-while – guarantees at least one execution
int attempts = 0;
do {
    attempts++;
} while (attempts < 3);

// enhanced for-loop ("for-each") – best for reading every element of a
collection
int[] values = {4, 8, 15, 16, 23, 42};
for (int v : values) {
    System.out.println(v);
}
```

`break` exits a loop immediately; `continue` skips to the next iteration. Both are used heavily in DSA code — for example, breaking out of a search loop the moment a target is found.

Exercises: Part I

1. **FizzBuzz:** print numbers 1 to 30. For multiples of 3 print “Fizz”, for multiples of 5 print “Buzz”, for multiples of both print “FizzBuzz”.
2. Write a loop that computes the factorial of a number entered via `Scanner`.
3. Using nested `for` loops, print a multiplication table from 1×1 to 5×5.
4. Modify the Collatz example above to also print the sequence of values, not just the step count.

Part II: Core Building Blocks for DSA

9. Arrays

An array is a fixed-size, ordered, index-based container of elements of the same type — the foundation of nearly every data structure you will study.

```
int[] nums = new int[5];      // all elements default to 0
nums[0] = 10;
nums[1] = 20;

int[] literals = {1, 2, 3, 4, 5}; // declare + initialize directly

System.out.println(nums.length); // 5 – note: length is a field, not a method
System.out.println(literals[2]);  // 3 – zero-indexed
```

Traversal:

```
for (int i = 0; i < literals.length; i++) {
    System.out.println(literals[i]);
}
```

Key facts that matter for DSA:

- Array size is fixed at creation; Java does not let you “resize” an array in place (this is exactly why `ArrayList`, covered in Part IV, exists).
- Access by index, `arr[i]`, is $O(1)$ — constant time — because arrays are stored as contiguous memory blocks.
- Insertion or deletion in the middle of an array requires shifting subsequent elements, which is $O(n)$.
- Accessing an out-of-range index throws `ArrayIndexOutOfBoundsException` at runtime, not a compile error.

```
int[] arr = {1, 2, 3};
System.out.println(arr[5]); // throws ArrayIndexOutOfBoundsException
```

10. Multi-Dimensional Arrays

```
int[][] grid = new int[3][4]; // 3 rows, 4 columns, all zero
grid[0][0] = 1;
grid[2][3] = 99;

int[][] matrix = {
    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 9}
};

for (int row = 0; row < matrix.length; row++) {
    for (int col = 0; col < matrix[row].length; col++) {
        System.out.print(matrix[row][col] + " ");
    }
    System.out.println();
}
```

2D arrays are the natural representation for grids, adjacency matrices in graph problems, and dynamic-programming tables.

Exercise 2.1: Given a 2D array representing a grid of 0s and 1s, write a method that counts how many cells contain a 1.

11. Strings and StringBuilder

`String` in Java is an **immutable** reference type: once created, its contents never change. Every operation that appears to “modify” a string actually returns a brand-new `String` object.

```
String s = "hello";
String upper = s.toUpperCase(); // "HELLO" – s itself is unchanged
System.out.println(s);          // still "hello"

System.out.println(s.length()); // 5
System.out.println(s.charAt(1)); // 'e'
System.out.println(s.substring(1, 3)); // "el"
System.out.println(s.indexOf('l')); // 2
System.out.println(s + " world"); // concatenation: "hello world"
System.out.println(s.equals("hello")); // true – use .equals(), never == ,
for content comparison
```

Why `.equals()` and not `==`? `==` on reference types compares memory addresses, not contents. Two different `String` objects with identical text can be unequal by `==` even though `.equals()` correctly reports them as equal. This is one of the most common early Java bugs.

Building strings efficiently: because `String` is immutable, repeatedly concatenating strings in a loop (`result = result + piece;`) creates a new object every iteration — $O(n^2)$ total work for n concatenations. `StringBuilder` solves this by using a mutable, resizable internal buffer:

```
StringBuilder sb = new StringBuilder();
for (int i = 0; i < 5; i++) {
    sb.append(i).append(",");
}
System.out.println(sb.toString()); // "0,1,2,3,4,"
sb.reverse(); // handy for palindrome-style problems
```

Exercise 2.2: Write a method `isPalindrome(String s)` that returns true if `s` reads the same forwards and backwards, ignoring case.

12. Methods and Parameter Passing

```
public class MathUtils {
    // static method: belongs to the class itself, callable without an instance
    static int square(int n) {
        return n * n;
    }

    public static void main(String[] args) {
        System.out.println(square(6)); // 36
    }
}
```

Method anatomy: `[modifiers] returnType methodName(parameterList) { body }.`

Parameter passing in Java is always “pass by value.” For primitives, a copy of the value is passed. For reference types (arrays, objects), a copy of the *reference* is passed — meaning the method can modify the object the reference points to, but reassigning the parameter itself inside the method has no effect outside it.

```

static void tryToDouble(int x) {
    x = x * 2;           // only changes the local copy
}

static void modifyArray(int[] arr) {
    arr[0] = 999;        // modifies the actual array, since arr points to
the same object
}

public static void main(String[] args) {
    int num = 5;
    tryToDouble(num);
    System.out.println(num); // 5 - unchanged

    int[] data = {1, 2, 3};
    modifyArray(data);

    System.out.println(data[0]); // 999 - changed, because both references point to
the same array
}

```

Understanding this distinction is essential before writing recursive algorithms that mutate arrays in place (e.g., in-place sorting).

13. Recursion

A recursive method calls itself on a smaller version of the same problem, until it reaches a **base case** that stops the recursion.

```

static int factorial(int n) {
    if (n == 0) return 1; // base case
    return n * factorial(n - 1); // recursive case
}

```

Every recursive method needs:

1. At least one **base case** that does not recurse.
2. A **recursive case** that moves strictly closer to the base case.

Tracing the call stack for `factorial(4)`:

```
factorial(4) = 4 * factorial(3)
factorial(3) = 3 * factorial(2)
factorial(2) = 2 * factorial(1)
factorial(1) = 1 * factorial(0)
factorial(0) = 1    <- base case reached, stack unwinds
```

Recursion on arrays — summing a range:

```
static int sum(int[] arr, int index) {
    if (index == arr.length) return 0;    // base case
    return arr[index] + sum(arr, index + 1); // recursive case
}
```

A classic double-recursion example — Fibonacci:

```
static int fib(int n) {
    if (n <= 1) return n;
    return fib(n - 1) + fib(n - 2);
}
```

This naive version recomputes the same subproblems repeatedly (exponential time); Part V revisits it with memoization for efficiency, foreshadowing dynamic programming.

Stack overflow: every recursive call consumes stack memory. A base case that is unreachable, or recursion that is too deep (commonly beyond tens of thousands of frames), throws `StackOverflowError`. This is why some recursive DSA algorithms are rewritten iteratively with an explicit `Stack` (Part IV) for very large inputs.

Exercises: Part II

1. Write a recursive method to compute the greatest common divisor (GCD) of two integers using the Euclidean algorithm.
2. Write an iterative and a recursive version of “reverse an array in place,” and compare them.
3. Write `countVowels(String s)` using a `for` loop, then rewrite it recursively.
4. Using `StringBuilder`, write a method that removes all whitespace from a string.

Part III: Object-Oriented Programming for DSA

Object-oriented programming (OOP) is how Java lets you build your own data structures (nodes, trees, graphs) rather than relying only on arrays.

14. Classes and Objects

```
public class Point {  
    int x;  
    int y;  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Point p = new Point(); // create an object (instance) of Point  
        p.x = 3;  
        p.y = 4;  
        System.out.println(p.x + ", " + p.y);  
    }  
}
```

A **class** is a blueprint; an **object** is a concrete instance created from that blueprint with the `new` keyword. Every object lives on the heap, and a variable of the class type is a reference to it — this connects directly back to the pass-by-reference behavior discussed in section 12.

15. Constructors and the `this` Keyword

```
public class Point {
    int x;
    int y;

    // Constructor: runs automatically when "new Point(...)" is called
    Point(int x, int y) {
        this.x = x; // "this.x" is the field; "x" is the parameter
        this.y = y;
    }
}

Point origin = new Point(0, 0);
```

If you write no constructor at all, Java supplies a no-argument **default constructor** automatically. As soon as you write any constructor yourself, that automatic default disappears — a frequent early source of confusion.

16. Encapsulation

Encapsulation means keeping a class's internal fields private and exposing controlled access through public methods (getters/setters). This protects invariants — for example, ensuring a custom `Stack` never reports a negative size.

```
public class BankAccount {
    private double balance; // "private" – not directly accessible outside this
class

    public BankAccount(double initialBalance) {
        this.balance = initialBalance;
    }

    public double getBalance() {
        return balance;
    }

    public void deposit(double amount) {
        if (amount > 0) balance += amount; // validation lives in one place
    }
}
```

17. Inheritance

Inheritance lets one class (a **subclass**) reuse and extend the fields and methods of another (a **superclass**), using `extends`.

```
class Animal {
    String name;
    Animal(String name) { this.name = name; }
    void speak() { System.out.println(name + " makes a sound."); }
}

class Dog extends Animal {
    Dog(String name) { super(name); } // calls Animal's constructor
    @Override
    void speak() { System.out.println(name + " barks."); }
}

Animal a = new Dog("Rex");
a.speak(); // "Rex barks." – the Dog version runs, even though the variable type
is Animal
```

Every class in Java implicitly extends `Object` if no other superclass is named, as described in Oracle's *Java Tutorials* lesson on inheritance (docs.oracle.com/javase/tutorial/java/IandI/subclasses.html).

18. Polymorphism

The example above — `a.speak()` calling `Dog`'s overridden version through an `Animal`-typed reference — is **runtime polymorphism**. It is what allows DSA code to be written generically: a method that accepts a `List<Shape>` can process `Circle`, `Square`, or any other subclass uniformly, calling the correct overridden behavior for each.

Method overloading (same method name, different parameter lists) is a second, compile-time form of polymorphism:

```
static int add(int a, int b) { return a + b; }
static double add(double a, double b) { return a + b; }
```

19. Abstraction: Abstract Classes and Interfaces

```

abstract class Shape {
    abstract double area(); // no body – every concrete subclass must implement
this

    void describe() {
        System.out.println("This shape has area " + area());
    }
}

class Circle extends Shape {
    double radius;
    Circle(double radius) { this.radius = radius; }
    @Override
    double area() { return Math.PI * radius * radius; }
}

```

An **interface** defines a contract of method signatures with no implementation (before Java 8) that any implementing class must fulfill:

```

interface Comparable2<T> {
    int compareTo(T other);
}

class Money implements Comparable2<Money> {
    int cents;
    Money(int cents) { this.cents = cents; }
    @Override
    public int compareTo(Money other) {
        return Integer.compare(this.cents, other.cents);
    }
}

```

This is exactly the pattern used by Java's real `Comparable` interface, revisited in Part IV, which powers sorting throughout the Collections Framework.

Abstract class vs. interface — when to use which:

	Abstract class	Interface
Fields	Can have instance fields	Only constants (<code>static final</code>)
Multiple inheritance	A class can extend only one abstract class	A class can implement many interfaces
Use when	Subclasses share common state/behavior	You need a pure capability contract (e.g., <code>Comparable</code> , <code>Iterable</code>)

20. Generics

Generics let a class or method operate on a type specified by the caller, while the compiler still enforces type safety — this is exactly how `ArrayList<T>` , `HashMap<K,V>` , and every other Collections Framework class work.

```

class Box<T> {
    private T content;
    void set(T content) { this.content = content; }
    T get() { return content; }
}

Box<String> stringBox = new Box<>();
stringBox.set("hello");
String value = stringBox.get(); // no cast needed – the compiler already knows
it's a String

Box<Integer> intBox = new Box<>();
intBox.set(42);

```

Without generics, older Java code stored everything as `Object` and required manual, unsafe casting — Oracle's *Java Tutorials* Generics lesson describes generics as adding compile-time type safety that eliminates this casting and catches many bugs before the program ever runs (docs.oracle.com/javase/tutorial/extra/generics/).

Exercises: Part III

1. Model a `Person` class with `name` and `age` fields, a constructor, and a `toString()` override. Create a `Student` subclass that adds a `gpa` field.
2. Define an interface `Shape` with an `area()` method, and implement it for `Rectangle` and `Triangle` .
3. Write a generic class `Pair<A, B>` that stores two values of possibly different types and has getters for each.

4. Explain, in your own words, why `ArrayList<int>` is not legal Java but `ArrayList<Integer>` is (hint: revisit Part V's section on wrapper classes after reading it).
-

Part IV: The Java Collections Framework

The Collections Framework is a unified set of interfaces and classes for storing and manipulating groups of objects. For DSA work, this is the single most important part of the Java standard library: it gives you production-quality, well-tested implementations of the exact structures a DSA course teaches you to build from scratch.

21. Overview of Collections

At the top, the framework splits into two main hierarchies:

- **Collection** interface (and its children **List**, **Set**, **Queue**) — groups of individual elements.
- **Map** interface — key-value pairs (technically separate from **Collection**, but considered part of the framework).

```
Collection
├── List      (ordered, allows duplicates)  -> ArrayList, LinkedList
├── Set      (no duplicates)               -> HashSet, LinkedHashSet,
TreeSet
├── Queue/Deque (FIFO / LIFO / double-ended) -> ArrayDeque, LinkedList,
PriorityQueue
Map (separate hierarchy, key -> value)      -> HashMap, LinkedHashMap,
TreeMap
```

All import from `java.util`. In generic code, always **program to the interface**, not the implementation:

```
List<Integer> list = new ArrayList<>(); // preferred style
```

This means you can swap `ArrayList` for `LinkedList` later without changing any code that only calls `List` methods.

22. List: ArrayList and LinkedList

`ArrayList` is backed internally by a resizable array; `LinkedList` is backed by a doubly-linked list of nodes.

```

import java.util.ArrayList;
import java.util.List;

List<String> names = new ArrayList<>();
names.add("Alice");
names.add("Bob");
names.add(1, "Carol");    // insert at index 1
System.out.println(names); // [Alice, Carol, Bob]
names.remove("Bob");
System.out.println(names.get(0)); // "Alice"
System.out.println(names.size()); // 2
System.out.println(names.contains("Carol")); // true

for (String n : names) {
    System.out.println(n);
}

```

Complexity comparison (from the behavior documented for each class in the Java SE API):

Operation	<code>ArrayList</code>	<code>LinkedList</code>
Get by index	O(1)	O(n)
Add/remove at end	Amortized O(1)	O(1)
Add/remove at front or middle	O(n) (shifts elements)	O(1) once positioned, O(n) to reach the position
Memory overhead	Lower (contiguous array)	Higher (node + two pointers per element)

Rule of thumb: default to `ArrayList` unless you specifically need frequent insertions/removals at the front or in the middle of a large collection, or you're implementing something like a deque.

23. Stack and Deque

A **stack** is Last-In-First-Out (LIFO). The legacy `java.util.Stack` class exists, but Oracle's own API documentation recommends `Deque` (specifically `ArrayDeque`) as the modern, faster replacement for stack operations.

```
import java.util.ArrayDeque;
import java.util.Deque;

Deque<Integer> stack = new ArrayDeque<>();
stack.push(1);
stack.push(2);
stack.push(3);
System.out.println(stack.pop()); // 3 – last in, first out
System.out.println(stack.peek()); // 2 – look without removing
```

Stacks are the natural structure for: balanced-parenthesis checking, undo functionality, depth-first search (DFS), and evaluating expressions.

```
// Classic DSA exercise: check balanced parentheses using a stack
static boolean isBalanced(String s) {
    Deque<Character> stack = new ArrayDeque<>();
    for (char c : s.toCharArray()) {
        if (c == '(' || c == '[' || c == '{') {
            stack.push(c);
        } else {
            if (stack.isEmpty()) return false;
            char open = stack.pop();
            if ((c == ')' && open != '(') ||
                (c == ']' && open != '[') ||
                (c == '}' && open != '{')) {
                return false;
            }
        }
    }
    return stack.isEmpty();
}
```

24. Queue and PriorityQueue

A **queue** is First-In-First-Out (FIFO) — essential for breadth-first search (BFS) and task scheduling.


```
import java.util.LinkedList;
import java.util.Queue;

Queue<Integer> queue = new LinkedList<>();
queue.offer(1);
queue.offer(2);
queue.offer(3);
System.out.println(queue.poll()); // 1 – first in, first out
System.out.println(queue.peek()); // 2
```

`PriorityQueue` orders elements by priority (natural ordering, or a supplied `Comparator`) rather than insertion order — the backbone of algorithms like Dijkstra’s shortest path and heap-based sorting.

```
import java.util.PriorityQueue;

PriorityQueue<Integer> minHeap = new PriorityQueue<>(); // smallest element has
highest priority by default
minHeap.offer(5);
minHeap.offer(1);
minHeap.offer(3);
System.out.println(minHeap.poll()); // 1

PriorityQueue<Integer> maxHeap = new PriorityQueue<>((a, b) -> b - a); //
reverse the ordering for a max-heap
```

25. Set: HashSet, LinkedHashSet, TreeSet

A `Set` stores unique elements — no duplicates.

```
import java.util.HashSet;
import java.util.Set;

Set<Integer> seen = new HashSet<>();
seen.add(5);
seen.add(5); // ignored – already present
System.out.println(seen.size()); // 1
System.out.println(seen.contains(5)); // true
```

Implementation	Ordering	Typical use
<code>HashSet</code>	No guaranteed order	Fastest general-purpose set; used to check “have I seen this before?” in $O(1)$ average time
<code>LinkedHashSet</code>	Preserves insertion order	When you need uniqueness <i>and</i> predictable iteration order
<code>TreeSet</code>	Sorted order	When you need elements in sorted order, or range queries (<code>headSet</code> , <code>tailSet</code>)

26. Map: HashMap, LinkedHashMap, TreeMap

`Map` stores key-value pairs and is arguably the single most-used structure in practical DSA problem solving (frequency counting, memoization, adjacency lists for graphs).

```
import java.util.HashMap;
import java.util.Map;

Map<String, Integer> wordCount = new HashMap<>();
String[] words = {"a", "b", "a", "c", "b", "a"};
for (String w : words) {
    wordCount.put(w, wordCount.getOrDefault(w, 0) + 1);
}
System.out.println(wordCount); // {a=3, b=2, c=1} (order not guaranteed for
HashMap)

for (Map.Entry<String, Integer> entry : wordCount.entrySet()) {
    System.out.println(entry.getKey() + " -> " + entry.getValue());
}
```

According to the official Java SE API documentation for `HashMap`, the class provides constant-time performance for the basic `get` and `put` operations, assuming the hash function spreads elements properly across buckets, and it grows (reshashes) automatically once the number of stored entries exceeds the product of its capacity and load factor — 0.75 by default (Oracle, *HashMap* — `java.util`, docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/HashMap.html).

Implementation	Ordering	Typical use
<code>HashMap</code>	No guaranteed order	Default choice; average $O(1)$ get/put
<code>LinkedHashMap</code>	Insertion order (or access order, configurable)	LRU-cache-style problems
<code>TreeMap</code>	Sorted by key	Range queries, ordered iteration by key, $O(\log n)$ operations

Exercise 4.1: Given an array of integers, use a `HashMap` to find the first repeated element in a single pass.

27. Comparable and Comparator

Java offers two ways to define how objects should be ordered, which underpins every sort you'll run in DSA code (`Collections.sort` , `Arrays.sort` , `TreeSet` , `TreeMap` , `PriorityQueue`).

```

import java.util.*;

class Student implements Comparable<Student> {
    String name;
    int score;
    Student(String name, int score) { this.name = name; this.score = score; }

    @Override
    public int compareTo(Student other) {
        return Integer.compare(this.score,
other.score); // natural ordering: by score ascending
    }

    @Override
    public String toString() { return name + "(" + score + ")"; }
}

List<Student> students = new ArrayList<>(List.of(
    new Student("Amit", 72), new Student("Bea", 90), new Student("Cy", 65)
));

Collections.sort(students); // uses compareTo – natural ordering
System.out.println(students); // [Cy(65), Amit(72), Bea(90)]

// Comparator: an alternative ordering, defined outside the class, without
modifying it
students.sort(Comparator.comparing((Student s) -> s.name));
System.out.println(students); // sorted alphabetically by name instead

```

Comparable defines a class’s single “natural” ordering (implemented inside the class). **Comparator** defines an ordering externally, and you can create as many different comparators as you need without touching the class itself.

Exercises: Part IV

1. Implement a queue using two stacks (a classic interview question).
2. Use a `HashMap<Character, Integer>` to find the first non-repeating character in a string.
3. Use a `TreeMap` to print a `Map<String, Integer>`’s entries sorted alphabetically by key.
4. Given a list of `Student` objects, sort them by score descending, then by name ascending for ties, using a chained `Comparator`.

Part V: Advanced Java for DSA

28. Exception Handling

```
try {
    int[] arr = new int[3];
    System.out.println(arr[5]); // triggers an exception
} catch (ArrayIndexOutOfBoundsException e) {
    System.out.println("Caught: " + e.getMessage());
} finally {
    System.out.println("This always runs, error or not.");
}
```

Exceptions matter in DSA code for validating input to your own data-structure implementations — for example, throwing `NoSuchElementException` or a custom exception when `pop()` is called on an empty stack, rather than letting the program fail unpredictably.

```
class EmptyStackException2 extends RuntimeException {
    EmptyStackException2(String message) { super(message); }
}
```

29. Wrapper Classes and Autoboxing

Generics (`List<T>`, `Map<K,V>`, etc.) cannot use primitive types directly — only objects. Java's **wrapper classes** (`Integer`, `Double`, `Boolean`, `Character`, `Long`, and so on) wrap a primitive in an object, which is why you write `List<Integer>` rather than `List<int>`.

```
Integer boxed = 5;           // autoboxing: primitive int automatically wrapped
into an Integer object
int unboxed = boxed;         // auto-unboxing: back to a primitive

List<Integer> list = new ArrayList<>();
list.add(10); // autoboxed to Integer automatically
int first = list.get(0); // auto-unboxed automatically
```

Caution: comparing wrapper objects with `==` compares references, not values, for values outside the small cached range (-128 to 127 for `Integer`). Always use `.equals()` (or `Integer.compare(a, b)`) when comparing wrapper values for equality.

30. Big-O Notation and Complexity in Java

Big-O notation describes how an algorithm's running time (or memory) grows as input size `n` grows. It is the language DSA uses to compare solutions independent of hardware.

Notation	Name	Example
$O(1)$	Constant	Array access by index, <code>HashMap</code> get/put (average case)
$O(\log n)$	Logarithmic	Binary search, balanced tree operations (<code>TreeMap</code> , <code>TreeSet</code>)
$O(n)$	Linear	Scanning an array once, linear search
$O(n \log n)$	Linearithmic	Efficient comparison sorts (<code>Collections.sort</code> , merge sort)
$O(n^2)$	Quadratic	Nested loops over the same array (naive bubble/insertion sort)
$O(2^n)$	Exponential	Naive recursive Fibonacci without memoization

Applying it to Part II's naive Fibonacci: without memoization, `fib(n)` recomputes overlapping subproblems and runs in exponential time. Adding a `HashMap` cache turns it into linear time:

```
static Map<Integer, Long> memo = new HashMap<>();

static long fibMemo(int n) {
    if (n <= 1) return n;
    if (memo.containsKey(n)) return memo.get(n);
    long result = fibMemo(n - 1) + fibMemo(n - 2);
    memo.put(n, result);
    return result;
}
```

This single change — using a `Map` to remember work already done — is the essence of **dynamic programming**, and it is one of the most important techniques you will carry forward from this guide into a full DSA course.

31. Bit Manipulation

Low-level bitwise operators appear often in DSA problems involving sets, flags, and optimization.

```
int a = 5;    // 0101
int b = 3;    // 0011

System.out.println(a & b);    // 1    (AND)
System.out.println(a | b);    // 7    (OR)
System.out.println(a ^ b);    // 6    (XOR)
System.out.println(~a);       // -6   (NOT)
System.out.println(a << 1);    // 10   (left shift = multiply by 2)
System.out.println(a >> 1);    // 2    (right shift = divide by 2)

// Common trick: check if a number is a power of two
static boolean isPowerOfTwo(int n) {
    return n > 0 && (n & (n - 1)) == 0;
}
```

32. Lambda Expressions and Streams (Brief Introduction)

Modern Java (8+) supports functional-style code, which you will encounter reading real-world Java and in Comparator definitions (as seen in section 27).

```
import java.util.*;
import java.util.stream.*;

List<Integer> numbers = List.of(1, 2, 3, 4, 5, 6, 7, 8);

// Lambda expression: a compact, unnamed function
Comparator<Integer> descending = (x, y) -> y - x;

// Streams: a pipeline for filtering/transforming collections
List<Integer> evenSquares = numbers.stream()
    .filter(n -> n % 2 == 0)
    .map(n -> n * n)
    .collect(Collectors.toList());

System.out.println(evenSquares); // [4, 16, 36, 64]
```

This is optional for early DSA practice but useful to recognize when reading solutions others have written.

Exercises: Part V

1. Rewrite the naive `fib` from Part II with memoization, as shown above, and print how many recursive calls each version makes for `n = 30`.
 2. Write a method using bitwise operators to count the number of set bits (1s) in an integer's binary representation.
 3. Implement a simple custom exception `InvalidCapacityException` and throw it from a custom array-backed stack if a negative capacity is requested.
 4. Use a stream to find the sum of squares of all odd numbers in a `List<Integer>`.
-

Part VI: Bridging Java to DSA Practice

33. Choosing the Right Data Structure

If you need to...	Use
Access elements by index frequently	<code>ArrayList</code> or array
Insert/remove frequently at both ends	<code>ArrayDeque</code>
Maintain LIFO order (undo, DFS, parsing)	<code>ArrayDeque</code> as a stack
Maintain FIFO order (BFS, task queues)	<code>ArrayDeque</code> or <code>LinkedList</code> as a queue
Always retrieve the smallest/largest element	<code>PriorityQueue</code>
Check membership / remove duplicates fast	<code>HashSet</code>
Maintain sorted, unique elements	<code>TreeSet</code>
Map keys to values, fast lookup	<code>HashMap</code>
Map keys to values, sorted by key	<code>TreeMap</code>
Count frequencies	<code>HashMap<T, Integer></code>
Cache expensive recursive results	<code>HashMap</code> (memoization)

34. Implementing a Stack and Queue from Scratch

Before relying on `java.util`, every DSA course has you build these yourself once, to understand what is happening underneath.

```
public class ArrayStack {  
    private int[] data;  
    private int top;  
  
    public ArrayStack(int capacity) {  
        data = new int[capacity];  
        top = -1;  
    }  
  
    public void push(int value) {  
        if (top == data.length - 1) throw new RuntimeException("Stack  
overflow");  
        data[++top] = value;  
    }  
  
    public int pop() {  
        if (isEmpty()) throw new RuntimeException("Stack underflow");  
        return data[top--];  
    }  
  
    public int peek() {  
        if (isEmpty()) throw new RuntimeException("Stack is empty");  
        return data[top];  
    }  
  
    public boolean isEmpty() {  
        return top == -1;  
    }  
}
```

```

public class LinkedList {
    private static class Node {
        int value;
        Node next;
        Node(int value) { this.value = value; }
    }

    private Node head, tail;

    public void enqueue(int value) {
        Node node = new Node(value);
        if (tail == null) {
            head = tail = node;
        } else {
            tail.next = node;
            tail = node;
        }
    }

    public int dequeue() {
        if (head == null) throw new RuntimeException("Queue is empty");
        int value = head.value;
        head = head.next;
        if (head == null) tail = null;
        return value;
    }
}

```

This `Node`-based pattern is exactly what you will reuse to build a singly linked list, a binary tree (two child references instead of one `next`), and eventually a graph (a list of neighbor references per node).

35. Worked DSA Problems in Java

Problem 1 — Binary search (requires a sorted array, runs in $O(\log n)$):

```

static int binarySearch(int[] arr, int target) {
    int low = 0, high = arr.length - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (arr[mid] == target) return mid;
        if (arr[mid] < target) low = mid + 1;
        else high = mid - 1;
    }
    return -1; // not found
}

```

Problem 2 — Two Sum using a `HashMap` for $O(n)$ time instead of the naive $O(n^2)$ nested loop:

```
static int[] twoSum(int[] nums, int target) {
    Map<Integer, Integer> seen = new HashMap<>(); // value -> index
    for (int i = 0; i < nums.length; i++) {
        int complement = target - nums[i];
        if (seen.containsKey(complement)) {
            return new int[] { seen.get(complement), i };
        }
        seen.put(nums[i], i);
    }
    return new int[] {};
}
```

Problem 3 — Breadth-first traversal of a graph represented as an adjacency list (`Map<Integer, List<Integer>>`), using a `Queue` and a `HashSet` for visited tracking:

```
static List<Integer> bfs(Map<Integer, List<Integer>> graph, int start) {
    List<Integer> order = new ArrayList<>();
    Set<Integer> visited = new HashSet<>();
    Queue<Integer> queue = new LinkedList<>();

    queue.offer(start);
    visited.add(start);

    while (!queue.isEmpty()) {
        int node = queue.poll();
        order.add(node);
        for (int neighbor : graph.getOrDefault(node, new ArrayList<>())) {
            if (!visited.contains(neighbor)) {
                visited.add(neighbor);
                queue.offer(neighbor);
            }
        }
    }

    return order;
}
```

36. Real-World Applications

- `HashMap`-based **caching** underlies web session stores, database query caches, and the memoization technique from Part V.
- `PriorityQueue` powers real routing/navigation systems (shortest-path algorithms), operating-system task schedulers, and event simulation.

- **Trees built from the Node pattern in section 34** underlie file systems, DOM/HTML parsing, and database indexes (B-trees).
 - **Graphs and BFS/DFS** model social networks, recommendation engines, and dependency resolution (e.g., build systems, package managers).
 - **ArrayDeque-based stacks** appear in compilers/parsers (matching brackets, expression evaluation) and browser back/forward history.
-

Summary and Key Takeaways

- Java's **static typing**, **automatic memory management**, and richly documented **Collections Framework** make it a strong language for learning DSA, and one of the most common languages in technical interviews.
- **Arrays** are the foundation of everything: $O(1)$ index access, but $O(n)$ insertion/deletion in the middle, and a fixed size once created.
- **Strings are immutable**; use `StringBuilder` for repeated concatenation to avoid $O(n^2)$ behavior.
- Parameters are always **pass by value** in Java — for reference types, that value is the reference itself, which is why mutating an object inside a method is visible outside it, but reassigning the parameter is not.
- **Recursion** needs a reachable base case and a recursive case that shrinks the problem; naive recursive solutions (like Fibonacci) can be exponential without **memoization**, a `HashMap`-based technique that is the gateway to dynamic programming.
- **OOP** (classes, inheritance, interfaces, generics) is how you build your own reusable structures — nodes, trees, graphs — and how the Collections Framework itself is built.
- The **Collections Framework** gives you production-ready `List`, `Set`, `Queue` / `Deque`, and `Map` implementations. Programming to the interface (`List<T> list = new ArrayList<>();`) keeps your code flexible.
- **Big-O notation** is the shared vocabulary for comparing algorithm efficiency; knowing the complexity of each Collections Framework operation (section 22, 26) lets you predict your own algorithm's performance before you run it.
- Once comfortable with Parts I–V, Part VI's pattern — pick the right structure, implement it once from scratch to understand it, then use the standard library version in practice — is exactly how a full DSA course, and most interview preparation, is structured.

Further Reading and References

Official / authoritative sources used and recommended for continued study:

1. Oracle, *The Java Tutorials* (Trail: Getting Started) — docs.oracle.com/javase/tutorial/getStarted/
2. Oracle, *The Java Tutorials* (Trail: Learning the Java Language) — docs.oracle.com/javase/tutorial/java/
3. Oracle, *The Java Tutorials* (Generics lesson) — docs.oracle.com/javase/tutorial/extra/generics/
4. Oracle, *Java SE Documentation* (overview and downloads) — docs.oracle.com/en/java/
5. Oracle, *Java SE API Specification* — [java.util.HashMap](https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/HashMap.html) — docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/HashMap.html
6. Oracle, *Java SE API Specification* (browse any class in [java.util](https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/) for its documented complexity guarantees, e.g. [ArrayList](https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/ArrayList.html), [TreeMap](https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/TreeMap.html), [PriorityQueue](https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/PriorityQueue.html), [ArrayDeque](https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/ArrayDeque.html))
7. Dev.java (Oracle's actively maintained modern Java learning site, successor content to the JDK-8-era tutorials) — dev.java

Suggested next steps after this guide:

- Work through a structured DSA problem set (arrays and strings first, then hash maps, then trees/graphs, then dynamic programming) implementing each solution in Java rather than pseudocode.
- Study time/space complexity proofs formally — this guide introduces Big-O intuitively; a DSA-specific text will formalize it.
- Once comfortable with linear structures, move to trees (binary search trees, heaps — the [PriorityQueue](https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/PriorityQueue.html) internals) and then graphs (BFS/DFS, shortest paths, minimum spanning trees).
- Read the source code of [java.util.ArrayList](https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/ArrayList.html) and [java.util.HashMap](https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/HashMap.html) (available in the JDK source, or on OpenJDK's repository) to see production-quality implementations of the ideas in Parts II and IV.